TEMPORADA XIV

CI NICARAGUA · 2026

DOCUMENTO DE LECTURA

02

// Unidad I · Fundamentos de Java

OPERADORES Y EXPRESIONES

CÓMO CALCULA Y DECIDE JAVA

EN ESTA LECTURA

01 ¿Qué es una expresión?
02 Operadores aritméticos
03 Asignación e incremento
04 Operadores de comparación
05 Operadores lógicos
06 Precedencia
07 Conversión de tipos
08 La clase Math



ELABORADO POR

MARVIN RAFAEL MEZA PINEDA

Facilitador · Centro Nacional de Innovación y Tecnología
Francisco “El Chele” Moreno

Fundamentos de Desarrollo
Backend con Java
Octubre 2026

CAPÍTULO 01

1 ¿QUÉ ES UNA EXPRESIÓN?

> Una **expresión** es cualquier combinación de valores, variables y operadores que Java puede **evaluar** para obtener un único resultado.

Cada vez que escribes `precio * cantidad` o `edad >= 18`, le pides a Java que haga un cálculo y te entregue un valor. Las piezas de una expresión tienen nombre:



- **Operandos:** los valores con los que se trabaja (variables, números, textos).
- **Operador:** el símbolo que indica qué hacer con ellos (`+`, `==`, `&&`…).
- **Resultado:** siempre es **un solo valor** y siempre tiene un **tipo**.

Toda expresión tiene un tipo

Este detalle es la clave de todo el documento. El tipo del resultado depende de los operandos y del operador:

Expresión	Resultado	Tipo del resultado
`5 + 3`	`8`	`int`: entero con entero da entero
`5.0 + 3`	`8.0`	`double`: si hay un decimal, el resultado es decimal
`5 > 3`	`true`	`boolean`: toda comparación da verdadero o falso
`"Hola " + "Ana"`	`"Hola Ana"`	`String`: unir textos da otro texto

Java usa cinco familias de operadores, que veremos en orden: **aritméticos** (calcular), **de asignación** (guardar), **de comparación** (comparar), **lógicos** (combinar condiciones) y la **conversión de tipos** (transformar un valor de un tipo a otro).

CAPÍTULO 02

2 OPERADORES ARITMÉTICOS

> Son los operadores de las matemáticas de siempre, con un par de comportamientos propios de la programación que debes conocer bien.

Operador	Operación	Ejemplo	Resultado
`+`	Suma	`7 + 2`	`9`
`-`	Resta	`7 - 2`	`5`
`*`	Multiplicación	`7 * 2`	`14`
`/`	División	`7 / 2`	`3` (¡no 3.5!)
`%`	Módulo (residuo)	`7 % 2`	`1`

La división entera

Cuando divides **dos enteros**, Java devuelve un **entero**: corta la parte decimal sin redondear. Por eso `7 / 2` da `3` y `9 / 10` da `0`. Para obtener decimales, al menos uno de los operandos debe ser decimal:

```
int a = 7, b = 2;
System.out.println(a / b);           // 3    -> int / int = int
System.out.println(7.0 / 2);         // 3.5  -> double / int = double
System.out.println((double) a / b);  // 3.5  -> convertimos a antes de dividir
```

ERROR CLÁSICO

`double promedio = (80 + 75 + 90) / 3;` guarda `81.0` y no `81.67`. La división se hace **entre enteros** antes de guardarse en el `double`. La solución es dividir entre `3.0`.

El operador módulo %

El módulo devuelve lo que **sobra** de una división entera. Parece poco útil, pero aparece en muchísimos programas:

Uso	Expresión	¿Por qué funciona?
¿Es par o impar?	`n % 2 == 0`	Los pares no dejan residuo al dividir entre 2.
¿Es múltiplo de 5?	`n % 5 == 0`	Mismo principio con cualquier número.
Último dígito	`1234 % 10` → `4`	El residuo entre 10 es la unidad.
Minutos a horas	`135 / 60` → `2` `135 % 60` → `15`	135 minutos son 2 horas y 15 minutos.

El + también une textos

Si uno de los operandos de `+` es un `String`, Java no suma: **concatena**. Y como las expresiones se evalúan de izquierda a derecha, el orden importa:

```java
// texto + 10 da texto; luego se le pega el 5
System.out.println("Total: " + 10 + 5);     // Total: 105

// los paréntesis hacen que se sume primero
System.out.println("Total: " + (10 + 5));   // Total: 15

// de izquierda a derecha: primero suma, luego une
System.out.println(10 + 5 + " pesos");      // 15 pesos
```

DATO

Cada tipo tiene un límite. Si a un `int` en su valor máximo (`2147483647`) le sumas 1, no da error: "da la vuelta" y se vuelve negativo. A esto se le llama **desbordamiento** (*overflow*). Para números muy grandes usa `long`.

CAPÍTULO 03

3 ASIGNACIÓN E INCREMENTO

> El signo `=` en Java **no significa "igual"**: significa "guarda el valor de la derecha en la variable de la izquierda".

Por eso `contador = contador + 1` tiene sentido en programación aunque en matemáticas sea imposible. Java primero calcula la derecha (`contador + 1`) y después guarda el resultado en `contador`. Como esta operación es tan común, existen formas abreviadas:

Operador	Ejemplo	Equivale a	Si x valía 10, ahora vale
`=`	`x = 10`	asignación simple	`10`
`+=`	`x += 5`	`x = x + 5`	`15`
`-=`	`x -= 3`	`x = x - 3`	`7`
`*=`	`x *= 2`	`x = x * 2`	`20`
`/=`	`x /= 4`	`x = x / 4`	`2` (división entera)
`%=`	`x %= 3`	`x = x % 3`	`1`

Incremento ++ y decremento --

`x++` suma 1 y `x--` resta 1. Los verás en casi todos los ciclos `for`. Pueden escribirse **antes** o **después** de la variable, y cuando forman parte de una expresión más grande la diferencia importa:

POSTFIJO · x++

```java
int x = 5;
int y = x++;
// 1) y recibe 5 (el valor actual)
// 2) luego x sube a 6
// y = 5, x = 6
```

PREFIJO · ++x

```java
int x = 5;
int y = ++x;
// 1) x sube a 6 primero
// 2) luego y recibe 6
// y = 6, x = 6
```

RECOMENDACIÓN

Cuando `++` va solo en su línea (`contador++;`) o en un `for`, prefijo y postfijo hacen exactamente lo mismo. Evita mezclarlos dentro de expresiones más grandes: el código se vuelve difícil de leer.

CAPÍTULO 04

4 OPERADORES DE COMPARACIÓN

Comparan dos valores y **siempre** devuelven un `boolean`: `true` o `false`. Son el corazón de los `if` y de los ciclos.

Operador	Significado	Ejemplo (edad = 17)	Resultado
`==`	igual a	`edad == 17`	`true`
`!=`	distinto de	`edad != 17`	`false`
`>`	mayor que	`edad > 18`	`false`
`<`	menor que	`edad < 18`	`true`
`>=`	mayor o igual que	`edad >= 14`	`true`
`<=`	menor o igual que	`edad <= 16`	`false`

= NO ES ==

Un solo `=` **guarda** un valor; dos `==` **comparan**. Si escribes `if (edad = 18)`, el compilador marcará error porque `edad = 18` no es una condición.

Comparar textos: usa .equals()

Con `String` el operador `==` **no compara el contenido**: compara si las dos variables apuntan al mismo objeto en memoria. A veces “funciona” por casualidad y otras no, sobre todo con textos que vienen del teclado. Para comparar el contenido de dos textos usa siempre `.equals()`:

```java
String respuesta = teclado.nextLine();      // el usuario escribe: si

// ✗ INCORRECTO: puede dar false aunque el usuario escribió "si"
if (respuesta == "si") { ... }

// ✓ CORRECTO: compara el contenido del texto
if (respuesta.equals("si")) { ... }

// ✓ Ignora mayúsculas: acepta "si", "Si" y "SI"
if (respuesta.equalsIgnoreCase("si")) { ... }
```

¿Y LOS DECIMALES?

Los `double` se guardan en binario y algunos valores no son exactos: `0.1 + 0.2 == 0.3` da `false` (el resultado real es `0.30000000000000004`). Para dinero en sistemas reales se usa la clase `BigDecimal`, que veremos más adelante en el curso.

CAPÍTULO 05

5 OPERADORES LÓGICOS

> Permiten **combinar** varias condiciones en una sola. Trabajan con valores `boolean` y devuelven un `boolean`.

Operador	Nombre	Devuelve true cuando…
`&&`	Y (AND)	**Ambas** condiciones son verdaderas.
`||`	O (OR)	**Al menos una** de las condiciones es verdadera.
`!`	NO (NOT)	La condición es falsa: **invierte** el valor.

Tablas de verdad

Muestran el resultado para todas las combinaciones posibles. Piensa en **A** = “tiene 14 años o más” y **B** = “aprobó 3er año”:

A	B	A && B	A || B	!A
`true`	`true`	**`true`**	**`true`**	`false`
`true`	`false`	`false`	**`true`**	`false`
`false`	`true`	`false`	**`true`**	**`true`**
`false`	`false`	`false`	`false`	**`true`**

Para entrar al curso se necesitan los dos requisitos, así que la condición correcta es `A && B`: si falta cualquiera de los dos, el resultado es `false`.

En código

```java
int edad = 16;
boolean aproboTercero = true;
int faltas = 2;

boolean puedeInscribirse = edad >= 14 && aproboTercero;      // true
boolean necesitaAviso    = faltas > 3 || edad < 14;          // false
boolean noPuede          = !puedeInscribirse;                // false

if (edad >= 14 && aproboTercero && faltas <= 3) {
    System.out.println("Inscripción aprobada");
}
```

Evaluación en cortocircuito

Java es eficiente: deja de evaluar en cuanto ya conoce el resultado.

- Con `&&`: si la primera condición es `false`, el resultado ya es `false` y **no revisa la segunda.**
- Con `||`: si la primera condición es `true`, el resultado ya es `true` y **no revisa la segunda.**

Esto permite escribir condiciones "protegidas", donde la primera parte evita que la segunda provoque un error:

```java
int divisor = 0;
if (divisor != 0 && 100 / divisor > 5) {    // como divisor != 0 es false,
    System.out.println("Mayor que 5");      // nunca se ejecuta 100 / 0
}                                           // y el programa no se cae
```

Negar una condición compuesta

Al aplicar `!` a una condición con `&&` u `||`, el operador se invierte (leyes de De Morgan). Las dos líneas de cada par son equivalentes:

```java
!(a && b)   es lo mismo que   !a || !b
!(a || b)   es lo mismo que   !a && !b

// "No es cierto que (tenga 14 años Y haya aprobado 3er año)"
// = "Tiene menos de 14 años O no aprobó 3er año"
!(edad >= 14 && aproboTercero)   ==   edad < 14 || !aproboTercero
```

CAPÍTULO 06

6 PRECEDENCIA DE OPERADORES

> Cuando una expresión tiene varios operadores, Java no la lee simplemente de izquierda a derecha: sigue un **orden de prioridad**, igual que en matemáticas la multiplicación va antes que la suma.

Orden	Operadores	Descripción
1	`( )`	Paréntesis: siempre primero
2	`x++  x--`	Incremento y decremento postfijo
3	`++x  --x  !  -x  (tipo)`	Unarios: prefijo, negación, signo, casting
4	`*  /  %`	Multiplicación, división, módulo
5	`+  -`	Suma y resta (y concatenación)
6	`<  >  <=  >=`	Comparación de orden
7	`==  !=`	Igualdad
8	`&&`	Y lógico
9	`||`	O lógico
10	`? :`	Operador ternario
11	`=  +=  -=  *=  /=  %=`	Asignación: siempre al final

Cuando dos operadores tienen la misma prioridad (por ejemplo `*` y `/`), se evalúan de **izquierda a derecha**. La asignación es la excepción: se evalúa de derecha a izquierda.

Paso a paso

```java
int r = 2 + 3 * 4 - 10 / 5;
//      2 + 12    - 2          -> primero * y /
//      14        - 2          -> luego + y -, de izquierda a derecha
//      12                     -> por último, se guarda en r

boolean ok = 5 + 3 > 7 && 10 % 3 == 1;
//           8     > 7 && 1      == 1   -> aritméticos primero
//           true      && true          -> después las comparaciones
//           true                       -> al final el &&
```

&& tiene más prioridad que ||

Esta regla causa errores de lógica muy difíciles de ver, porque el programa compila y corre, pero decide mal:

```
// Intención: "es estudiante o docente, Y además está activo"

// ✗ Java lo lee como: esEstudiante || (esDocente && activo)
if (esEstudiante || esDocente && activo)

// ✓ Los paréntesis dicen exactamente lo que queremos
if ((esEstudiante || esDocente) && activo)
```

REGLA DE ORO

No hace falta memorizar la tabla completa. **Cuando dudes, usa paréntesis.** No cambian la velocidad del programa y hacen que cualquier persona entienda tu intención al leer el código.

CAPÍTULO 07

7 CONVERSIÓN DE TIPOS

A veces necesitas usar un valor de un tipo como si fuera de otro: un entero como decimal, un decimal como entero, o un texto como número.

Conversión automática (ampliación)

Java convierte solo cuando el cambio es **seguro**, es decir, de un tipo "pequeño" a uno "grande" donde el valor siempre cabe:




```
int entero = 25;
double decimal = entero;       // automático: decimal vale 25.0
long grande = entero;          // automático: grande vale 25
```

Casting explícito (reducción)

En la dirección contraria se puede **perder información**, así que Java te obliga a decirlo de forma explícita escribiendo el tipo destino entre paréntesis. Esto se llama **casting**:

```
double precio = 99.95;
int precioEntero = (int) precio;      // 99: se CORTA el decimal

int total = 250, personas = 3;
double porPersona = (double) total / personas;  // 83.33...
long redondeado = Math.round(99.95);   // 100: para redondear, usa Math
```

CUIDADO

`(int) 3.99` da `3`, no `4`. El casting a entero **trunca**: elimina los decimales sin importar cuán cerca esté del siguiente número.

De texto a número y de número a texto

Un `String` no se puede convertir con casting. Para eso existen métodos especiales, muy útiles cuando los datos llegan como texto (desde un formulario, un archivo o una API):

Método	Ejemplo	Resultado
`Integer.parseInt(texto)`	`Integer.parseInt("25")`	`25` (int)
`Double.parseDouble(texto)`	`Double.parseDouble("9.5")`	`9.5` (double)
`Boolean.parseBoolean(texto)`	`Boolean.parseBoolean("true")`	`true` (boolean)
`String.valueOf(valor)`	`String.valueOf(25)`	`"25"` (String)

Si el texto no representa un número válido, por ejemplo `Integer.parseInt("hola")`, el programa se detiene con un error `NumberFormatException`. Aprenderemos a manejar estos errores más adelante.

CAPÍTULO 08

8 LA CLASE MATH

> Java no tiene un operador para potencias ni para raíces. Para eso usa la clase `Math`, que viene incluida y **no necesita import**.

Método	Qué hace	Ejemplo	Resultado
`Math.pow(a, b)`	Potencia: a elevado a b	`Math.pow(2, 3)`	`8.0`
`Math.sqrt(x)`	Raíz cuadrada	`Math.sqrt(25)`	`5.0`
`Math.abs(x)`	Valor absoluto	`Math.abs(-7)`	`7`
`Math.round(x)`	Redondea al entero más cercano	`Math.round(4.6)`	`5`
`Math.floor(x)`	Redondea hacia abajo	`Math.floor(4.9)`	`4.0`
`Math.ceil(x)`	Redondea hacia arriba	`Math.ceil(4.1)`	`5.0`
`Math.max(a, b)`	El mayor de dos valores	`Math.max(8, 3)`	`8`
`Math.min(a, b)`	El menor de dos valores	`Math.min(8, 3)`	`3`
`Math.random()`	Decimal aleatorio entre 0.0 y 1.0 (sin incluir 1)	`Math.random()`	`0.7341…`
`Math.PI`	Constante π	`Math.PI`	`3.14159…`

Juntando todo

Los métodos de `Math` se combinan con los operadores que ya conoces. Dos casos muy comunes:

```
// Área de un círculo de radio 3:  π · r²
double radio = 3;
double area = Math.PI * Math.pow(radio, 2);          // 28.27...

// Número entero aleatorio entre 1 y 100
int aleatorio = (int) (Math.random() * 100) + 1;
//   Math.random()          -> 0.0 ... 0.999
//   * 100                  -> 0.0 ... 99.9
//   (int)                  -> 0 ... 99   (casting: corta el decimal)
//   + 1                    -> 1 ... 100
```

FÍJATE

`pow`, `sqrt`, `floor` y `ceil` devuelven `double`, aunque el resultado sea "redondo". Si necesitas un `int`, aplica casting: `int cubo = (int) Math.pow(3, 3);`

Resumen del documento

Familia	Operadores	Recuerda
Aritméticos	`+ - * / %`	Entero entre entero da entero. `%` devuelve el residuo.
Asignación	`= += -= *= /= %= ++ --`	`=` guarda, no compara.
Comparación	`== != > < >= <=`	Devuelven `boolean`. Textos con `.equals()`.
Lógicos	`&& || !`	Cortocircuito. `&&` va antes que `||`.
Conversión	`(tipo)  parseInt  valueOf`	El casting a entero corta, no redondea.

Referencias

Oracle. *The Java™ Tutorials: Operators*. docs.oracle.com/javase/tutorial/java/nutsandbolts/operators.html

Oracle. *Java SE 25 API: java.lang.Math*. docs.oracle.com/en/java/javase/25/docs/api



MARVIN RAFAEL MEZA PINEDA

Facilitador · Fundamentos de Desarrollo Backend con Java

Centro Nacional de Innovación y Tecnología Francisco “El Chele” Moreno · Temporada XIV · 2026